2026-5-31 C++map和set--理论部分

- 1. 容器的分类
  - 1.1 序列式容器
  - 1.2 关联式容器
- 2. set的使用
  - 2.1 set类的介绍
  - 2.2 set的构造和迭代器
    - 2.3 set的增删查
  - multiset和set的差异
- 3. map的使用
  - 3.1 map类的使用
  - 3.2 pair类型介绍

1. 容器的分类

1.1 序列式容器

逻辑结构为线性序列的数据结构两个存储位置之间没有紧密的关联，哪怕说位置交换一下依旧是序列式容器。
如：list,vector,array等

1.2 关联式容器

关联式式容器与序列式容器不同的是，关联式容器逻辑结构通常是非线性结构，两个文件有紧密的关联联系，交换一下他的存储结构就别破坏了。顺序容器中的元素是按关键字来保存和访问的。

2. set的使用

2.1 set类的介绍

class template

std::set

<set>

```
template < class T,                        // set::key_type/value_type
           class Compare = less<T>,        // set::key_compare/value_compare
           class Alloc = allocator<T>      // set::allocator_type
           > class set;
```

Set

Sets are containers that store unique elements following a specific order.

In a `set`, the value of an element also identifies it (the value is itself the *key*, of type `T`), and each value must be unique. The value of the elements in a `set` cannot be modified once in the container (the elements are always const), but they can be inserted or removed from the container.

Internally, the elements in a `set` are always sorted following a specific *strict weak ordering* criterion indicated by its internal comparison object (of type `Compare`).

`set` containers are generally slower than unordered_set containers to access individual elements by their *key*, but they allow the direct iteration on subsets based on their order.

Sets are typically implemented as *binary search trees*.

1. set默认要求T的是小于，如果你想比较大于的话可以写仿函数传给第二个模板参数
2. set底层存储数据的内存是从空间配置器申请的，如果需要可以自己实现内存池传递给第三个参数。
3. set的底层是用红黑树实现的，增删改查数据的时间复杂度为$\log_2(N)$，迭代器遍历是走的搜索树的中序。

2.2 set的构造和迭代器

1. set支持正向和反向迭代器遍历，遍历默认按照升序排列，因其底层是二叉搜索树，所以迭代器遍历走的是中序遍历。
2. 这就意味着支持范围for,但需要注意的是**set的iterator和const_iterator都不支持迭代器修改数据**，修改关键字数据，破坏了底层搜索树结构。

2.3 set的增删查

multiset和set的差异

multise和set的使用基本相同，主要的区别在于multiset支持值冗余，那么二者围绕着insert/find/count/erase都具有一些差异。

3. map的使用

3.1 map类的使用

std::**map** <map>

```
template < class Key,                                     // map::key_type
           class T,                                       // map::mapped_type
           class Compare = less<Key>,                     // map::key_compare
           class Alloc = allocator<pair<const Key,T> >    // map::allocator_type
           > class map;
```

Map

Maps are associative containers that store elements formed by a combination of a *key value* and a *mapped value*, following a specific order.

In a `map`, the *key values* are generally used to sort and uniquely identify the elements, while the *mapped values* store the content associated to this *key*. The types of *key* and *mapped value* may differ, and are grouped together in member type `value_type`, which is a pair type combining both:

```
typedef pair<const Key, T> value_type;
```

Internally, the elements in a `map` are always sorted by its *key* following a specific *strict weak ordering* criterion indicated by its internal comparison object (of type `Compare`).

`map` containers are generally slower than unordered_map containers to access individual elements by their *key*, but they allow the direct iteration on subsets based on their order.

The mapped values in a map can be accessed directly by their corresponding key using the *bracket operator* ((operator[]).

Maps are typically implemented as *binary search trees*.

1. Key是map底层的关键字类型
2. T是map底层value类型，set默认支持Key支持小于比较，如果不支持或者需要大于比较的话可以自行实现仿函数传给第二个模板参数
3. map底层的空间是从空间配置器申请的
4. map的底层是用红黑树实现的，增删改查数据的时间复杂度为$\log_2(N)$，迭代器遍历是走的搜索树的中序,所以是按照key有序遍历的。

3.2 pair类型介绍